

Project Report

Description

What was the initial idea / background of the project?

The initial idea for this project came from an interest in applying functional programming techniques to a small but realistic problem. A workout tracker was chosen as the domain because it is simple enough to implement in a limited timeframe, while still being rich enough to motivate meaningful design decisions around types, parsing, and API design.

The primary focus was never the user interface or the feature set, but rather how the problem could be modelled cleanly using functional techniques. The core goals were:

- Design a strongly typed domain model for workouts, exercises, and sets.
- Implement a small DSL for workout inputs using monadic parsing.
- Expose a type-safe web API where correctness is enforced at compile time.

Workouts were to be stored in an in-memory structure initially, keeping the scope manageable and allowing focus to remain on the functional design rather than infrastructure. A database layer was considered as a potential extension if time permitted.

The motivation was to build something where functional programming techniques were not just incidental, but central to how the system works. Types prevent invalid states, the parser gives structured error handling, and the API is verified by the compiler rather than by runtime tests.

What were the goals of the project?

The goals of the project followed directly from the initial idea. The technical goals were to demonstrate meaningful application of functional programming techniques rather than building a large feature set. This meant:

- A working web API where routes and handlers are verified at compile time through Servant.
- A functional DSL parser built with MegaParsec that gives structured error messages on invalid input.
- A clean monad transformer stack separating configuration, error handling, and IO concerns.
- Property-based tests with QuickCheck to verify domain invariants and parser correctness.

How did the project turn out?

The project turned out well and met all the initial goals. The core components, the domain model, DSL parser, web API, and monad transformer stack were all implemented as planned and work together cleanly.

Beyond the initial scope, several features were added that were not part of the original proposal. A minimal web frontend was added to make the system easier to interact with. The frontend consists of a single static `index.html` file, which is served directly by the Haskell server through Servant. When a user opens <http://localhost:8080>, the server reads and delivers the file from disk. The HTML and JavaScript in the frontend were generated with the assistance of an AI tool, which I considered acceptable given the primary focus was the Haskell implementation. The frontend exists purely to make the system more accessible for interaction, and the interesting technical work is all contained in the Haskell codebase. Having a browser interface simply made it easier to test and demonstrate the system without needing to use curl for every interaction. A progress tracking feature was added that accumulates training volume per exercise across sessions and displays it as a chart. Workouts can also be deleted from the history.

The DSL was extended beyond the original design. Instead of requiring each set to be written on a separate line, the parser was updated to support a grouped syntax where multiple sets of the same exercise can be written on one line:

Squat 10x100.0, 8x110.0, 6x120.0

The exercise type was also generalised from a fixed enum of three exercises to a generic newtype accepting any capitalised name, making the system open to any exercise without modifying the source code.

The main challenge encountered was getting the Servent API to correctly handle incoming requests. The server was running, but the routes were not responding, which turned out to be caused by the handler chain not matching the type-level API definition. Once the ordering was corrected, the issue was resolved. This was a good illustration of how Servant's type-level encoding catches structural mistakes at compile time.

Overall, the project ended up being a clean demonstration of how functional techniques can be applied to realistic problems, with the type system and abstractions actively contributing to correctness rather than being incidental to the implementation.

What future extensions can you imagine for the project?

There are several extensions I can imagine for this project. I have already considered a Database through the Selda library. Currently, all workouts are stored in an `IORef` and are lost when the server restarts. Adding a database layer would make the system persistent across sessions. The monad

transformer stack was designed with this in mind. The AppEnv record holds the store, so swapping an IORef for a database connection would be largely contained to App.hs and Server.hs, without touching the domain model, parser, or API types. Another extension that might be relevant is to make richer statistics. Statistics such as personal records, average volume per session or volume trends over time could be computed using the existing HasVolume typeclass and exposed through additional API endpoints.

Instructions

Running the project

Start the server by running the following command in the project root:

```
cabal run
```

The server will start locally, and the application can be accessed in your browser at

<http://localhost:8080>

Using the DSL

Workouts are submitted using a simple text format. Each exercise follows this pattern:

ExerciseName RepsxWeight

For example:

Bench 10x100.0

Squat 5x140.0

Deadlift 3x180.0

Exercise names must start with a capital letter. Weight is written as a decimal number. If you want to log multiple sets of the same exercise, you can either write them on separate lines:

Bench 10x100.0

Bench 8x110.0

Bench 6x120.0

Or you can use the shorthand comma syntax to group them on one line:

Bench 10x100.0, 8x110.0, 6x120.0

Both formats produce the same result.

Submitting and viewing the workouts

Once you have written your workouts, click Submit. The result panel will show a breakdown of each set and the total training volume for that session. Previous workouts are listed below the input field. Each entry shows the exercise performed, the total volume, and a delete button to remove it from the history.

Click the Progress tab to view a chart of your training volume over time. Each exercise you have logged appears as a button. Select one of these buttons to see how your volume for that exercise has developed across sessions.

A description of techniques

Algebraic Data Types And Typeclasses

Algebraic data types (ADTs) are a core feature of Haskell's type system. They allow you to define types by combining other types in two ways: product types (records, where a value contains multiple fields) and sum types (where a value is one of several alternatives). This gives you a precise way to model a domain where invalid states are impossible to represent.

Typeclasses are Haskell's mechanism for ad-hoc polymorphism, defining an interface that multiple types can implement independently. This is similar in spirit to interfaces in object-oriented languages, but more flexible since instances can be defined separately from the type definition.

The domain model is built entirely from ADTs in this project. Exercise is a newtype wrapping Text, Set is a product type grouping an exercise with repetitions and weight, and Workout is a newtype wrapping a list of sets. This means it is impossible to construct a Set without an exercise, repetitions, and a weight. The type system enforces the invariant at compile time, rather than at runtime.

A custom typeclass, HasVolume, was defined to abstract volume calculation across the different types. This allows volume to work uniformly on both a single set and a full workout. Any future type, such as a WeeklyPlan, can implement the same interface by adding an instance without modifying existing code.

ADTs were very well suited for this project. The domain is naturally hierarchical. A workout contains sets, each set has an exercise and measurements, and ADTs model this directly. The type system caught several mistakes during development that would have been runtime errors in a dynamically typed language. The HasVolume typeclass was a natural fit for abstracting volume calculation, and the separation between the typeclass definition and its instances kept the code modular and extensible.

Monadic Parsing with MegaParsec

Monadic parsing is an approach to building parsers using the Monad abstraction. A parser is modelled as a function that takes an input stream and returns either a result and the remaining input, or a parse error. Because parsers form a monad, can they be composed using `do` notation, making complex parsers easy to build from smaller, reusable pieces.

MegaParsec is a Haskell parsing library built on this idea. It provides combinators for common patterns, parsing literals, whitespace, and numbers, and produces detailed, structured error messages when parsing fails. It supports the `Applicative` and `Alternative` typeclasses, meaning parsers can be combined without explicit sequencing where the result of one parser does not depend on the value of another.

The DSL parser is implemented entirely with MegaParsec. The parser is built bottom-up from small, focused components. When a user submits invalid input, MegaParsec's `errorBundlePretty` produces a readable error message that is returned to the frontend as a 400 Bad Request.

MegaParsec was very well-suited for this project. The combinator style made it natural to build the parser incrementally, and the structured error messages are significantly better than what a hand-written parser would produce. The `Applicative` style (`*>` for sequencing without binding) kept the code concise where the result of one parser was not needed. The main limitation is that MegaParsec has a learning curve. Understanding many, `some`, `optional` and `eof` and how they interact takes time. But once understood, the library is expressive and reliable.

Type-Level Programming with Servant

Servant is a Haskell library for defining web APIs at the type level. Instead of describing routes as strings or configuration, the entire API is encoded as a single Haskell type using type operators. This means the compiler can verify at compile time that the server implementation matches the API specification. If a handler is missing, has the wrong type, or is in the wrong order, the code will not compile.

This is an example of type-level programming, where types carry information that is checked at compile time rather than runtime. The `:>` operator chains parts of a route together, and `:<|>` combines multiple routes into a single API type.

In my project, I define the full API as a single type:

type API

= Get '[HTML] Html

:<|> "workouts" :> Get '[JSON] [Workout]

:<|> "workouts" :> ReqBody '[PlainText] String :> Post '[JSON] WorkoutResponse

:<|> "workouts" :> Capture "index" Int :> Delete '[JSON] NoContent

:<|> "progress" :> Get '[JSON] [ProgressResponse]

Each line describes one endpoint, the path, the HTTP method, what it accepts, and what it returns.

Servant ensures that the server implementation matches this type exactly. During development, a bug was encountered where a handler was missing from the server, and the compiler caught it immediately as a type error rather than a runtime error.

hoistServer was used to convert handlers written in the custom App monad into Servant's Handler monad, keeping the handlers clean and decoupled from Servant's internal types:

server env = hoistServer api (appToHandler env) appServer

Servant was well-suited for this project focused on type safety. The compile-time guarantees meant that once the code compiled, the routing was correct by construction. The main challenge was the learning curve. Servant's type errors can be difficult to read, and understanding how hoistServer works requires careful study. However, the payoff was significant: the API type serves as both the implementation contract and the documentation, and mismatches between the frontend and backend were caught at compile time rather than discovered through testing.

Monad Transformers

Monad transformers are a technique for combining multiple monadic effects into a single monad. Each transformer adds one effect to an existing monad. ReaderT adds read-only access to an environment, ExceptT adds structured error handling, and IO provides input/output at the base. Stacking these transformers gives a single monad that supports all of their effects simultaneously, without having to thread state or errors manually through every function.

The resulting monad stack is typically wrapped in a newtype and given instances for the relevant typeclasses (MonadReader, MonadError, MonadIO), which makes the effects available through standard functions like asks, throwError, and liftIO.

The application monad is defined in App.hs:

```
newtype App a = App
  { runApp :: ReaderT AppEnv (ExceptT AppError IO) a }
  deriving
    ( Functor
    , Applicative
    , Monad
    , MonadReader AppEnv
    , MonadError AppError
    , MonadIO
    )
```

ReaderT AppEnv threads the application environment, currently an IORef holding the workout store, through every handler automatically. ExceptT AppError provides structured error handling with a domain-specific error type. IO sits at the base since the handlers need to read and write the store. This means the handlers can be written cleanly without manually passing the environment or wrapping the results in Either:

```
postWorkout :: String -> App WorkoutResponse

postWorkout input =
  case parse workoutParser "" input of
    Left err ->
      throwError $ ParseError (errorBundlePretty err)
    Right w -> do
      ref <- asks store
      liftIO $ modifyIORef ref (w :)
      return $ WorkoutResponse
        { workout = w
        , volume = Calc.volume w
        }
```

In `Server.hs`, `appToHandler` converts `App` into `Servant's Handler` by peeling off each transformer layer in order:

```
appToHandler :: AppEnv -> App a -> Handler a
```

```
appToHandler env app =
```

```
  runApp app
```

```
    & (runReaderT env)
```

```
    & runExceptT
```

```
    & liftIO
```

```
    >>= either (throwError . toServerError) return
```

```
  where (&) = flip ($)
```

Monad transformers were very well suited for structuring the application effects. The handlers are clean and focused on business logic; they never mention `Handler` or `ServerError` directly. The main cost is complexity: understanding how the layers interact and how to peel them off in `appToHandler` requires a solid understanding of monads. For a larger project, the transformer stack would likely be extended further, for example, adding a logging layer, and the design supports this without changing the handlers.

Property-Based Testing with QuickCheck

Property-based testing is a testing approach where, instead of writing specific test cases with known inputs and outputs, you define general properties that should hold for all valid inputs. The testing library generates hundreds of random inputs automatically and checks that the property holds for each one. If a counterexample is found, the library reports the failing input.

QuickCheck is the original property-based testing library for Haskell. It uses the `Arbitrary` typeclass to generate random values of any type, and properties are written as functions that return `Bool`. This approach is particularly powerful for testing parsers. A roundtrip check can see if a printed value can be parsed back to the original value, across a large space of random inputs.

`Arbitrary` instances were defined for `Exercise`, `Set`, and `Workout` to enable random generation:

```
instance Arbitrary Exercise where
```

```
  arbitrary = do
```

```
    first <- elements ['A' ... 'Z']
```

```
    rest <- listOf (elements(['a' ... 'z'] ++ ['A' ... 'Z']))
```

```
    return $ Exercise (T.pack (first : rest))
```


Several properties were defined covering domain invariants and parser correctness:

```
prop_totalVolumeNonNegative :: Workout -> Bool
```

```
prop_totalVolumeNonNegative w = Calc.volume w >= 0
```

```
prop_setVolume :: Set -> Bool
```

```
prop_setVolume s = Calc.volume s == fromIntegral (repetitions s) * weight s
```

```
prop_roundtrip :: Workout -> Bool
```

```
prop_roundtrip w = parse workoutParser "" (prettyWorkout w) == Right w
```

The roundtrip property is the most valuable; it simultaneously validates both the parser and the pretty printer across hundreds of randomly generated workouts.

Even though QuickCheck was demanded for this assignment, it was well-suited for testing the domain invariants and the parser. The roundtrip property, in particular, found subtle issues with the pretty printer during development that specific test cases would have missed. The main limitation is that Arbitrary instances need to generate values that satisfy the parser's constraints. For example, exercise names must start with a capital letter, which requires careful implementation of the generators. Overall, property-based testing provided significantly more confidence in the correctness of the parser than handwritten test cases alone would have.

Self evaluation

The project was a genuinely enjoyable experience. Working in Haskell on a realistic problem made the functional programming techniques feel motivated rather than academic; every design decision had a concrete reason behind it. Servant's type-level API encoding was particularly satisfying, since the compiler actively caught mistakes that would have been silent runtime bugs in other languages. Seeing the monad transformer stack come together and produce clean, readable handlers was also rewarding.

The iterative development process worked well. Starting with an MVP and building outward meant that each stage had a solid foundation to build on. The project ended up going beyond the initial scope in a natural way, with the frontend and progress tracking features adding real usability without compromising the core design.

With more time, the first thing to add would be database persistence through Selda. The in-memory store works for demonstration purposes, but losing all data on server restart is a significant limitation for a real application. The AppEnv was designed to make this swap straightforward, but it was not completed within the project timeframe.

The pretty printer could also be more robust. Currently, it groups consecutive sets with the same exercise, but if a workout interleaves exercises, for example, Bench, Squat, and Bench, the grouping does not merge the two Bench groups. This is correct behaviour for preserving workout order, but a more sophisticated pretty printer could handle this case more elegantly.

Finally, the frontend JavaScript could have been structured more carefully. Since it was generated with AI assistance and was not the primary focus of the project, some of the JavaScript is more verbose than necessary. If the project were to be extended seriously, the frontend would benefit from a proper framework or at least a cleaner separation of concerns.